Project Management Report

Distributed Financial Transaction Processor

Sampath Pranay Beela, Rahul Chinya Jagadeesha
CS6650 — Building Scalable Distributed Systems

Spring 2026

1. Initial design vs. final state

We began with a one-page sketch of the system: an HTTP API in front, a worker pool in the back, and a queue in between. The sketch answered "what shape is this going to be" but nothing else. The final system is a fully cloud-deployed, Terraform-provisioned, Locust-benchmarked, four-experiment-validated pipeline running on real AWS infrastructure. Table **??** captures the journey from intent to outcome.

Aspect	Initial intent (Week 1)	Final state (Week 8)
Architecture	API + worker + queue + DB	ALB → ECS Fargate api-svc → SQS (+ DLQ) → ECS Fargate worker-svc → DynamoDB (3 tables) + CloudWatch
Deployment target	"we'll figure out AWS later"	Terraform-managed stack in `us-west-2`, preflight-gated under LabRole
Concurrency control	optimistic only	optimistic + pessimistic, swap by env var, compared head-to-head
Correctness story	"we'll make it atomic"	single `TransactWriteItems` for debit + credit + status; crash-test-verified
Observability	console logs	CloudWatch log groups for both services + six Prometheus-style worker counters + JSON `METRICS` log lines every 30 s
Load testing	"some Locust script"	two user classes, four experiment runners, auto-rendered chart pipeline
Reproducibility	manual steps	every task a preflight-gated `make` target; every experiment a shell script that reseeds, runs, drains, verifies

Table 1: Where the project started vs. where it ended.

Three things changed the most between "initial intent" and "final state." First, we committed early to a **cloud-only** workflow rather than a local-compose-then-cloud workflow — this made the middle of the project harder but produced experimental numbers we could actually defend. Second, we decided to wrap the debit, credit, and status update in a single `TransactWriteItems` rather than three separate writes, which turned out to be the single design decision that the crash-recovery experiment proved correct. Third, we added a second concurrency-control strategy (pessimistic locking) specifically so Experiment 3 could answer a real question, not just measure our one implementation.

2. How we broke the problem down

We used the eight-week timeline in the project implementation plan as a living roadmap. Each of the four two-week phases owned a distinct set of deliverables with explicit dependencies so work could flow between phases without stalling. Table **??** is the phase breakdown we actually followed.

Phase	Deliverables	Success criteria
Weeks 1–2 *Foundations*	Architecture finalization; repo + service scaffolding; DynamoDB schema; core transfer workflow; API-side idempotency; SQS producer and consumer	One transfer can be submitted via HTTP and is applied exactly once to DynamoDB
Weeks 3–4 *Cloud + first load*	Dockerfiles + ECR; Terraform (ECR, ECS, ALB, SQS, Dynamo, CloudWatch); cloud smoke test; Locust harness (two workload classes); **Exp 1** (hot-account storm); **Exp 2** (worker crash recovery)	System runs on AWS end-to-end; two experiments produce reproducible numbers
Weeks 5–6 *Comparison + scale*	Pessimistic locking implementation; mode-swap harness; **Exp 3** (opt vs. pess on cloud); **Exp 4** (horizontal scaling 1:1, 2:2, 4:4)	Both modes characterized; scaling curve produced; correctness held at every config
Weeks 7–8 *Analysis + delivery*	Chart generator; summary-markdown generator; final report (LaTeX, 24 pages with embedded AWS screenshots); experiments report (5 pages); lessons-learned reflections; demo prep	Publication-ready artifacts produced; demo runnable end-to-end from a clean shell

Table 2: Four phases, each with a clear "we move on when this is true" gate.

Within each phase, we worked in short commit-sized units: each commit landed one self-contained piece (a handler, a script, a Terraform resource, an experiment runner) behind a feature flag or behind a preflight check, so neither of us ever blocked the other's work. This mattered the most during Weeks 3–4, when one of us was writing Terraform while the other was writing the crash-recovery experiment against the output of that Terraform.

3. Who worked on what

The team is two people. The work split came directly from the project implementation plan and held without revision through all eight weeks. Table **??** is the per-item RACI view.

We kept the split clean by designing around interfaces: the API handler (Pranay) called into DynamoDB accessors (Rahul) via a typed Go interface, and the two concurrency-control paths (Rahul's pessimistic, Pranay-integrated with the existing optimistic) lived behind the same `Processor.attemptTransfer` method. Neither of us had to wait for the other to finish a module before we could start ours.

4. Problems encountered and how they were resolved

We kept an informal issue log as we went. The entries that mattered most to the final state are summarized in Table **??**. Each entry is a real obstacle that cost at least one session to diagnose.

The issues cluster into three categories. The **tooling** category (Docker cross-build, shell quoting, macOS `date`, Locust venv) were one-time environment mismatches — annoying but solvable with one-liner changes. The **cloud-identity** category (LabRole, VPC) was about our account not matching the assumptions baked into the assignment; fixing these taught us more about AWS IAM trust policies than the rest of the project combined. The **correctness-under-concurrency** category

(reseed conflicts, drain waits, `ConsistentRead`) was the most educational — those were all places where the system was doing the right thing but our *observation* of the system was wrong, and fixing them required us to separate "is the system correct" from "can I see that it is correct."

5. Timeline of commits and milestones

We did not keep formal sprint retros, but every pull-request-sized unit of work maps to a phase milestone. At a high level the commit cadence looked like this: Weeks 1–2 was heavy on scaffolding commits and the first end-to-end transfer; Weeks 3–4 was dominated by Terraform + cloud-first deployment and the first two experiments; Weeks 5–6 was pessimistic locking plus the two comparison experiments; Weeks 7–8 was documentation, screenshots, chart rendering, and reports. The repository's commit history reflects that cadence — not the number of commits (the exact count is not the right metric), but the shape of them: feature commits early, infrastructure commits in the middle, refinement and analysis commits at the end.

6. What we would do differently next time

Three process-level changes we would make if we started over:

1. **Build the verification harness with the same care as the service.** Our transient-mismatch scare during Experiment 4 came from the verifier using an eventually consistent Scan, not from the system losing money. If we had treated `verify_balances.py` as first-class code from day one (with `ConsistentRead=True`, a drain gate, and proper error discrimination between "mismatch" and "still in flight"), we would have saved ourselves an hour of debugging a non-bug.
2. **Bootstrap the cloud identity story on day one.** We initially assumed a LabRole existed in our AWS account. It didn't. Creating LabRole ourselves (`scripts/bootstrap_labrole.sh`) should have happened during Weeks 1–2, not discovered halfway through Weeks 3–4. The general lesson: verify every external assumption your stack makes *before* you start depending on it.
3. **Cross-platform build defaults early.** A single-line `docker buildx --platform linux/amd64` in the original `push_images.sh` would have saved an entire deploy cycle during the first ECS rollout. We now treat architecture pinning as a day-one concern on any project that targets a cloud runtime.

7. Final state

Every deliverable in the project implementation plan is done. The system is deployable end-to-end via `make tf-apply && make push-images && make tf-apply && make cloud-seed && make cloud-smoke`, and tears down cleanly via `make tf-destroy`. Each experiment has its own `make` target that produces timestamped artifacts under `load-tests/experiments/`, including charts and a markdown summary. Four reports live under `load-tests/`: `final-report.pdf` (comprehensive individual report), `experiments-report.pdf` (5-page experiments summary), `lessons-learned.pdf` (individual reflection), and this project-management report. The full repository is self-contained and reproducible on any AWS account that has a LabRole-shaped role, with no elevated permissions required at apply time.

Work item	Owner	Artifact
Project scope and architecture finalization	Pranay & Rahul	`arch.md`, design discussions
Repository and service scaffolding	Pranay	`api-svc/`, `worker-svc/`
DynamoDB schema design	Rahul	`accounts`, `transactions`, `account_locks` tables
Core transfer workflow	Pranay & Rahul	API handler (Pranay), worker commit primitives (Rahul)
API-layer idempotency	Rahul	`PutTransactionIfNotExists` + handler short-circuit
SQS integration	Pranay	`api-svc/queue`, `worker-svc/queue`
Local correctness verification	Rahul	`load-tests/verify_balances.py`
Containerization	Pranay	multi-stage `Dockerfile`s, `scripts/push_images.sh`
Terraform infrastructure	Rahul	`infra/`
Cloud smoke testing	Pranay & Rahul	`scripts/smoke_test.sh`, `make cloud-smoke`
Load testing framework	Pranay	`load-tests/locustfile.py`, two user classes
Experiment 1 — hot-account storm	Pranay	`make cloud-test-load-hot`, results CSVs
Experiment 2 — worker crash recovery	Rahul	`scripts/crash_recovery_test.sh`
Pessimistic locking implementation	Rahul	`worker-svc/processor` pessimistic path, `account_locks` primitives
Experiment 3 — opt vs. pess	Pranay & Rahul	`scripts/compare_locking_modes.sh`
Experiment 4 — horizontal scaling	Pranay & Rahul	`scripts/scale_experiment.sh`
Data analysis and chart generation	Pranay	`scripts/render_charts.py`, `summarize_locust_results.py`
Final report writing	Rahul	`load-tests/final-report.tex`
Final demo preparation	Pranay & Rahul	demo script + recorded walkthrough

Table 3: Per-item ownership. "Pranay & Rahul" items were paired work where both of us contributed meaningfully.

Problem	Root cause	Resolution
ECS `CannotPullContainerEr` on first deploy	Images built on Apple Silicon (`arm64`); Fargate defaults to `linux/amd64`	Switched `scripts/push_images.sh` to `docker buildx build --platform linux/amd64`
`terraform apply` fails with "no matching EC2 VPC found"	No default VPC in `us-west-2` in this AWS account	`aws ec2 create-default-vpc --region us-west-2` once; unblocks all applies
`LabRole` not present in the lab account	Our Northeastern SSO account is not AWS Academy-shaped	Wrote `scripts/bootstrap_labrole.sh` to create the role and attach the four managed policies; tightened the preflight check to assert the role is ECS-assumable rather than assert the caller *is* LabRole
`aws iam create-role` failing with shell-quoting errors	`zsh` brace expansion eating multiline JSON arguments	Moved trust policy to `infra/labrole-trust.json`; every `aws iam` call now uses `file://`
`date +%s%3N` producing literal `...3N` on macOS	macOS `date` has no `%N` format	Replaced with `python3 -c 'import time; print(int(time.time()*1000))'` in both sweep scripts
Between-iteration reseed hitting `TransactionConflictEx`	In-flight worker `TransactWriteItems` hold a short lock on the item being `PutItem`'d by reseed	Added SQS purge + 65-second visibility drain before reseed; added 8-attempt exponential backoff retry in the seed script for `TransactionConflictException`
Transient balance mismatch during scaling sweep	Verifier ran before SQS queue fully drained; verifier used eventually consistent `Scan`	Added drain loop (visible + in-flight = 0) with 15-second settle buffer; switched `verify_balances.py` to `ConsistentRead=True`
One verify failure aborting whole sweep	`set -e` treated the transient mismatch as fatal	Sweep scripts now log a warning, write the verify text to a file, and continue to the next iteration
Locust venv import error (`zope.event`)	Stale dependency in shared venv	Rebuilt `/tmp/locust-venv` from scratch; added `boto3` so `verify_balances.py` runs in the same interpreter

Table 4: Nine issues we hit, each with root cause and resolution. Every one of these is discussed in more depth in `final-report.pdf`.